06

HAFTA

C# ile Nesneye Yönelik Programlama · Ders Notu

SOLID Prensipleri - I

(Temiz Kod için Temel Kurallar)

Bu Haftanın Kazanımları

Bu dersin sonunda öğrenciler:

- ✓ SOLID prensiplerinin neden ortaya çıktığını ve kötü tasarımın belirtilerini (**rigidity (katılık)**, **fragility (kırılganlık)**, **immobility (taşınamazlık)**) açıklar.
- ✓ **SRP (Single Responsibility Principle — Tek Sorumluluk İlkesi)**'ni uygulayarak birden fazla sorumluluk taşıyan sınıfları ayrıştırır.
- ✓ **OCP (Open/Closed Principle — Açık/Kapalı İlkesi)** ile yeni davranışları mevcut kodu değiştirmeden ekler.
- ✓ **LSP (Liskov Substitution Principle — Liskov Yerine Geçme İlkesi)** ihlallerini tanır ve kalıtım hiyerarşilerini doğru kurar.
- ✓ Üç prensibi küçük **refactoring (yeniden düzenleme)** senaryolarında uygular.

İçindekiler

1. SOLID Nedir? Neden Gerekir?
2. SRP (Single Responsibility)
3. OCP (Open/Closed)
4. LSP (Liskov Substitution)

— Haftanın Özeti

— Anahtar Kavramlar

— Alıştırmalar

— Ek Okuma Kaynakları

1. SOLID Nedir? Neden Gerekir?

Çalışan kod yazmak başlangıçtır; asıl zorluk, kodun aylar ve yıllar boyunca değişen gereksinimlere uyum sağlayabilmesidir. Robert C. Martin (Uncle Bob), 2000 yılındaki bir makalesinde nesne yönelimli tasarım için beş temel ilke derlemiş; bu ilkelerin baş harfleri daha sonra Michael Feathers tarafından **SOLID** kısaltmasıyla bir araya getirilmiştir.

Harf	Prensip	Tek cümlelik özet	Hafta
S	Single Responsibility Principle	Bir sınıfın değişmek için tek bir nedeni olmalı	6
O	Open/Closed Principle	Genişlemeye açık, değişime kapalı	6
L	Liskov Substitution Principle	Alt tipler üst tiplerin yerine sorunsuz kullanılabilmeli	6
I	Interface Segregation Principle	Kullanılmayan metotlara bağımlılık dayatılmamalı	7
D	Dependency Inversion Principle	Somut sınıflara değil soyutlamalara bağımlı olunmalı	7

1.1 Kötü tasarımın belirtileri

Martin, bozulmaya başlamış bir tasarımın kendini belirli belirtilerle gösterdiğini söyler. Bu belirtileri tanımak, SOLID'in hangi derde deva olduğunu anlamanın en kısa yoludur.

Belirti	Açıklama
Rigidity (katılık)	Küçük bir değişiklik, birbirine bağlı pek çok modülde zincirleme değişiklik gerektirir.
Fragility (kırılganlık)	Bir yerde yapılan değişiklik, mantıksal olarak ilgisiz başka yerleri bozar.
Immobility (taşınamazlık)	Bir parçayı başka bir projede yeniden kullanmak, onu çevresinden ayırmak çok zor olduğu için imkânsız hâle gelir.
Viscosity (akışkanlık direnci)	Doğru olanı yapmak, "hızlı yama" yapmaktan daha zordur; ekip kestirme yollara yönelir.

Bu belirtilerin ortak kökü iki kavramdır: **coupling (bağımlılık)** ve **cohesion (uyum)**. İyi bir tasarımda modüller arası bağımlılık düşük (**loose coupling — gevşek bağlılık**), modülün kendi içindeki uyum ise yüksektir (**high cohesion — yüksek uyum**). SOLID prensiplerinin tamamı bu iki hedefe hizmet eder.

> NOT **Prensipler kural değil, pusuladır**
>
> SOLID prensipleri körü körüne uygulanacak yasalar değildir. Amaç, değişimin maliyetini düşürmektir. Hiç değişmeyecek küçük bir betikte soyutlama katmanları kurmak, prensiplerin ruhuna aykırıdır.

2. SRP (Single Responsibility)

> TANIM **Tek Sorumluluk İlkesi**
>
> Bir sınıfın değişmek için yalnızca **bir nedeni** olmalıdır. Martin daha sonra tanımı şöyle netleştirmiştir: bir modül yalnızca tek bir **actor (aktör — paydaş grubu)** karşısında sorumlu olmalıdır.

"Tek sorumluluk", "tek metot" anlamına gelmez. Buradaki kilit kavram **reason to change (değişme nedeni)**'dir. Muhasebe birimi vergi kuralını, veritabanı yöneticisi tablo yapısını, pazarlama birimi e-posta

şablonunu değiştirmek isteyebilir. Bu üç farklı talep aynı sınıfı değiştiriyorsa, o sınıf üç farklı aktöre hizmet ediyor demektir.

2.1 Kötü örnek: her şeyi yapan Fatura sınıfı

C# · FaturaKotu.cs

```
public class Fatura
{
    public int Id { get; set; }
    public string MusteriEposta { get; set; } = "";
    public List<FaturaKalemi> Kalemler { get; } = [];

    // 1) İş kuralı: muhasebenin kararlarıyla değişir
    public decimal ToplamHesapla() =>
        Kalemler.Sum(k => k.BirimFiyat * k.Adet) * 1.20m;   // KDV %20

    // 2) Kalıcılık: veritabanı değişince değişir
    public void Kaydet()
    {
        using var baglanti = new SqlConnection("Server=...;Database=Muhasebe");
        // INSERT INTO Faturalar ...
    }

    // 3) Sunum: çıktı biçimi değişince değişir
    public string YazdirilabilirMetin() =>
        $"FATURA #{Id}\n" + string.Join("\n", Kalemler.Select(k => k.Ad));

    // 4) Bildirim: e-posta altyapısı değişince değişir
    public void EpostaGonder()
    {
        var smtp = new SmtpClient("smtp.firma.com");
        smtp.Send("muhasebe@firma.com", MusteriEposta,
                  "Faturanız", YazdirilabilirMetin());
    }
}
```

Bu sınıfın dört farklı değişme nedeni vardır. KDV oranını test etmek isteyen biri, sınıfın SQL ve SMTP kütüphanelerine de bağımlı olduğunu görür. İki farklı ekip aynı dosyayı aynı anda değiştirmek zorunda kalır ve birinin değişikliği diğerinin kodunu bozabilir: bu, kırılganlığın ta kendisidir.

2.2 Refactoring: sorumlulukları ayırmak

C# · Faturalyi.cs

```
public record FaturaKalemi(string Ad, decimal BirimFiyat, int Adet);

// Yalnızca fatura verisi ve ona ait temel hesap
public class Fatura
{
    public int Id { get; set; }
    public string MusteriEposta { get; set; } = "";
    public List<FaturaKalemi> Kalemler { get; } = [];
    public decimal AraToplam => Kalemler.Sum(k => k.BirimFiyat * k.Adet);
}

public class FaturaHesaplayici                  // aktör: muhasebe
{
    private const decimal KdvOrani = 0.20m;
    public decimal GenelToplam(Fatura f) => f.AraToplam * (1 + KdvOrani);
}
```

C# · Faturalyi.cs

```
public class FaturaRepository                    // aktör: veritabanı yönetimi
{
    public void Kaydet(Fatura f) { /* EF Core ile kayıt (8. hafta) */ }
}

public class FaturaYazici                        // aktör: raporlama / sunum
{
    public string Metin(Fatura f) =>
        $"FATURA #{f.Id}{Environment.NewLine}" +
        string.Join(Environment.NewLine,
                    f.Kalemler.Select(k => $"{k.Ad} x{k.Adet}"));
}

public class FaturaBildirimServisi               // aktör: müşteri iletişimi
{
    private readonly FaturaYazici _yazici;
    public FaturaBildirimServisi(FaturaYazici yazici) => _yazici = yazici;
    public void Gonder(Fatura f) { /* _yazici.Metin(f) ile e-posta gönder */ }
}
```

C# · Program.cs

```
var fatura = new Fatura { Id = 1, MusteriEposta = "ali@ornek.com" };
fatura.Kalemler.Add(new FaturaKalemi("Klavye", 750m, 2));
Console.WriteLine(new FaturaHesaplayici().GenelToplam(fatura));   // 1800,00
```

Artık KDV kuralı değiştiğinde yalnızca `FaturaHesaplayici` değişir ve bu sınıf veritabanı ya da e-posta altyapısı olmadan tek başına test edilebilir.

2.3 SRP ihlali nasıl fark edilir?

- Sınıfı tek cümleyle anlatırken "ve" bağlacına ihtiyaç duyuyorsanız ("faturayı hesaplar **ve** kaydeder **ve** gönderir").
- Sınıf adı `Manager`, `Helper`, `Utils`, `Processor` gibi belirsiz kelimeler içeriyorsa.
- Dosyanın başındaki `using` listesi birbiriyle ilgisiz altyapıları içeriyorsa (`System.Net.Mail`, `Microsoft.Data.SqlClient`, `System.Drawing`).
- Metotlar, sınıfın alanlarının farklı alt kümelerini kullanan gruplara ayrışıyorsa (düşük uyum).
- Versiyon kontrol geçmişinde dosya, farklı kişiler tarafından farklı nedenlerle sürekli değiştiriliyorsa.

> **DİKKAT** **Aşırı parçalamaktan kaçının**
>
> SRP'yi "her sınıfta tek metot" olarak yorumlamak, onlarca minik sınıf arasında kaybolmanıza yol açar. Birlikte değişen şeyler birlikte durmalı, farklı nedenlerle değişen şeyler ayrılmalıdır.

3. OCP (Open/Closed)

> **TANIM** **Açık/Kapalı İlkesi**
>
> Yazılım varlıkları (sınıflar, modüller, fonksiyonlar) **genişlemeye açık, değişime kapalı** olmalıdır. İlke, Bertrand Meyer tarafından 1988'de tanımlanmıştır. Yeni bir davranış, mevcut ve test edilmiş kodu değiştirerek değil, yeni kod ekleyerek kazandırılmalıdır.

3.1 Kötü örnek: büyüyen switch bloğu

C# · IndirimKotu.cs

```
public enum MusteriTipi { Standart, Ogrenci, Premium }

public class IndirimHesaplayici
{
    public decimal Hesapla(decimal tutar, MusteriTipi tip)
    {
        switch (tip)
        {
            case MusteriTipi.Standart: return 0m;
            case MusteriTipi.Ogrenci:  return tutar * 0.10m;
            case MusteriTipi.Premium:  return tutar * 0.15m + 20m;
            default: throw new ArgumentOutOfRangeException(nameof(tip));
        }
    }
    // Yeni müşteri tipi = enum + bu sınıf değişir, her şey yeniden test edilir
}
```

Her yeni müşteri tipi bu sınıfın açılıp değiştirilmesini gerektirir. Üstelik benzer `switch` blokları kodun başka yerlerinde de varsa, tek bir iş kuralı için onlarca dosyaya dokunmak gerekir. Bu kötü kokuya **shotgun surgery (saçma dağınık değişiklik)** denir.

3.2 Refactoring: soyutlama ve polymorphism

Çözüm, değişen kısmı (indirim kuralı) bir **abstraction (soyutlama)** arkasına almaktır. Her kural kendi sınıfında yaşar; hesaplayıcı yalnızca arabirimi tanır. Bu yapı, 5. haftadaki Strategy kalıbının doğrudan bir uygulamasıdır.

C# · IndirimIyi.cs

```
public record Musteri(string Ad, bool OgrenciMi = false, bool Premium = false,
                      string? VergiNo = null);

public interface IIndirimKurali
{
    bool Uygular(Musteri m);
    decimal Hesapla(decimal tutar);
}

public class OgrenciIndirimi : IIndirimKurali
{
    public bool Uygular(Musteri m) => m.OgrenciMi;
    public decimal Hesapla(decimal tutar) => tutar * 0.10m;
}

public class PremiumIndirimi : IIndirimKurali
{
    public bool Uygular(Musteri m) => m.Premium;
    public decimal Hesapla(decimal tutar) => tutar * 0.15m + 20m;
}

// YENİ kural: mevcut hiçbir sınıfa dokunmadan eklendi
public class KurumsalIndirim : IIndirimKurali
{
    public bool Uygular(Musteri m) => m.VergiNo is not null;
    public decimal Hesapla(decimal tutar) => tutar >= 5000m ? tutar * 0.20m : 0m;
}
```

C# · IndirimIyi.cs

```csharp
public class IndirimHesaplayici
{
    private readonly IEnumerable<IIndirimKurali> _kurallar;

    public IndirimHesaplayici(IEnumerable<IIndirimKurali> kurallar) =>
        _kurallar = kurallar;

    // En avantajlı indirimi uygular; yeni kurallar için bu metot DEĞİŞMEZ
    public decimal Hesapla(Musteri m, decimal tutar) =>
        _kurallar.Where(k => k.Uygular(m))
                 .Select(k => k.Hesapla(tutar))
                 .DefaultIfEmpty(0m)
                 .Max();
}
```

C# · Program.cs

```csharp
var hesaplayici = new IndirimHesaplayici(
    [new OgrenciIndirimi(), new PremiumIndirimi(), new KurumsalIndirim()]);

var ayse = new Musteri("Ayşe", OgrenciMi: true);
Console.WriteLine(hesaplayici.Hesapla(ayse, 1000m));   // 100,00
```

İPUCU **DI ile doğal uyum**

1. haftada göreceğimiz Dependency Injection ile birden fazla `IIndirimKurali` uygulaması kaydedilip `IEnumerable<IIndirimKurali>` olarak enjekte edilebilir. Yeni bir kural eklemek, yeni bir sınıf yazıp tek satırlık bir kayıt eklemekten ibaret olur.

3.3 OCP'yi sağlamanın yolları

Teknik	Açıklama	Örnek
Kalıtım + polymorphism	`abstract`/`virtual` üyeleri alt sınıflarda override etmek	`Sekil.AlanHesapla()`
Arabirim + Strategy	Değişen davranışı arabirim arkasına almak	`IIndirimKurali`
Delegate / lambda	Davranışı parametre olarak geçmek	`liste.Sort(karsilastirici)`
Decorator	Mevcut nesneyi sarmalayarak davranış eklemek	Loglama, önbellekleme katmanı
Extension member	Tipin koduna dokunmadan ona yeni üyeler kazandırmak	`metin.Kisalt(10)`

3.4 Tipi değiştirmeden genişletmek: extension members

extension method (genişletme metodu), kaynak koduna erişemediğiniz bir tipe (ör. `string`) yeni metotlar eklemenizi sağlar. Tip gerçekte değişmez; derleyici çağrıyı statik bir metoda yönlendirir.

YENİ **C# 14: extension blokları**

C# 14 ile gelen `extension` blokları sayesinde artık yalnızca metot değil, **extension property** ve statik extension üyeleri de tanımlanabilir.

C# · MetinUzantilari.cs

```csharp
public static class MetinUzantilari
{
    // Klasik extension method (C# 3'ten beri)
    public static string Kisalt(this string metin, int uzunluk) =>
        metin.Length <= uzunluk ? metin : metin[..uzunluk] + "…";

    // C# 14: extension bloğu ile property eklemek
    extension(string metin)
    {
        public bool BosMu => string.IsNullOrWhiteSpace(metin);
        public int KelimeSayisi =>
            metin.Split(' ', StringSplitOptions.RemoveEmptyEntries).Length;
    }
}
```

C# · Program.cs

```csharp
Console.WriteLine("Nesneye yönelik programlama".Kisalt(7));   // Nesneye…
Console.WriteLine("   ".BosMu);                               // True
Console.WriteLine("C# ile temiz kod".KelimeSayisi);           // 4
```

DİKKAT **Her şeyi baştan soyutlamayın**

OCP, "gelecekte olabilecek her değişikliğe karşı kod yazın" demek değildir. Değişimin gerçekten beklendiği noktaları koruyun. Pratik bir kural: bir yerde ilk kez değişiklik yapıyorsanız doğrudan değiştirin; aynı türden değişiklik ikinci kez geldiğinde soyutlamayı kurun.

4. LSP (Liskov Substitution)

TANIM **Liskov Yerine Geçme İlkesi**

S, T'nin bir alt tipiyse; programda T tipindeki nesneler, programın doğruluğunu bozmadan S tipindeki nesnelerle değiştirilebilmelidir. İlke, Barbara Liskov'un 1987'deki bir konuşmasına dayanır ve 1994'te Jeannette Wing ile birlikte biçimsel olarak tanımlanmıştır.

LSP, derleyicinin kontrol ettiği tip uyumluluğundan daha fazlasını ister: **davranışsal uyumluluk**. Bir alt sınıf, üst sınıfın arabirimini uygulamakla kalmamalı, üst sınıfı kullanan istemcinin makul beklentilerini de karşılamalıdır. "Is-a (bir ...dır)" ilişkisi yalnızca kavramsal olarak değil, davranış olarak da doğru olmalıdır.

4.1 Klasik ihlal: Kare bir Dikdörtgen midir?

C# · DikdortgenKare.cs

```
public class Dikdortgen
{
    public virtual int Genislik { get; set; }
    public virtual int Yukseklik { get; set; }
    public int Alan => Genislik * Yukseklik;
}

public class Kare : Dikdortgen      // matematikte "kare bir dikdörtgendir"
{
    public override int Genislik
    {
        get => base.Genislik;
        set { base.Genislik = value; base.Yukseklik = value; }
    }

    public override int Yukseklik
    {
        get => base.Yukseklik;
        set { base.Genislik = value; base.Yukseklik = value; }
    }
}
```

C# · Program.cs

```
AlanTesti(new Dikdortgen());   // Doğru
AlanTesti(new Kare());         // HATA! Alan = 16

static void AlanTesti(Dikdortgen d)
{
    d.Genislik = 5;
    d.Yukseklik = 4;           // istemci: "genişlik 5'te kalır" diye varsayar
    Console.WriteLine(d.Alan == 20 ? "Doğru" : $"HATA! Alan = {d.Alan}");
}
```

Matematikte her kare bir dikdörtgendir; ancak **değiştirilebilir** bir yazılım modelinde `Kare`, `Dikdortgen`'in davranış sözleşmesini bozar: genişliği ayarlamak yüksekliği de değiştirir. Çözüm, iki şekli ortak bir soyutlamanın kardeşleri yapmak ve tercihen değişmez tasarlamaktır:

C# · Sekiller.cs

```
public interface ISekil
{
    double Alan { get; }
}

public record Dikdortgen(double Genislik, double Yukseklik) : ISekil
{
    public double Alan => Genislik * Yukseklik;
}

public record Kare(double Kenar) : ISekil
{
    public double Alan => Kenar * Kenar;
}
```

4.2 LSP'nin kuralları: sözleşmeye dayalı tasarım

LSP, Bertrand Meyer'in **design by contract (sözleşmeye dayalı tasarım)** yaklaşımıyla birlikte düşünüldüğünde somut kurallara dönüşür:

Kural	Anlamı	İhlal örneği
Precondition (ön koşul) güçlendirilemez	Alt sınıf, üst sınıfın kabul ettiği girdileri reddetmemeli	Üst sınıf her tutarı kabul ederken alt sınıfın 100 TL altını reddetmesi
Postcondition (son koşul) zayıflatılamaz	Alt sınıf, üst sınıfın garanti ettiği sonucu sağlamalı	Üst sınıf asla `null` döndürmezken alt sınıfın `null` döndürmesi
Invariant (değişmez) korunmalı	Nesne için her zaman geçerli olan kurallar bozulmamalı	"Bakiye negatif olamaz" kuralının alt sınıfta çiğnenmesi
Yeni istisna türü eklenmemeli	Üst tipin sözleşmesinde olmayan hata fırlatılmamalı	`NotSupportedException`, `NotImplementedException`

4.3 Gerçek hayattan bir ihlal ve çözümü

C# · OdemeKotu.cs

```
public abstract class OdemeYontemi
{
    public abstract void Ode(decimal tutar);
    public abstract void IadeEt(decimal tutar);
}

public class HediyeCeki : OdemeYontemi
{
    public override void Ode(decimal tutar) { /* çekten düş */ }

    public override void IadeEt(decimal tutar) =>
        throw new NotSupportedException("Hediye çeki iade edilemez!"); // ihlal!
}
```

`OdemeYontemi` alan ve `IadeEt` çağıran her istemci, `HediyeCeki` geldiğinde çöker. İstemcinin kendini korumak için `if (yontem is HediyeCeki)` gibi tip kontrolleri yazmaya başlaması, LSP ihlalinin en belirgin işaretidir. Çözüm, yetenekleri ayrı arabirimlere bölmektir (7. haftadaki ISP'ye bir giriş):

C# · OdemeIyi.cs

```
public interface IOdemeYontemi { void Ode(decimal tutar); }
public interface IIadeEdilebilir { void IadeEt(decimal tutar); }

public class KrediKarti : IOdemeYontemi, IIadeEdilebilir
{
    public void Ode(decimal tutar) { /* bankaya çekim isteği */ }
    public void IadeEt(decimal tutar) { /* bankaya iade isteği */ }
}

public class HediyeCeki : IOdemeYontemi
{
    public void Ode(decimal tutar) { /* çekten düş */ }
}

// İade işlemi yalnızca iade edilebilen yöntemleri kabul eder;
// hata artık çalışma zamanında değil, derleme zamanında yakalanır.
public class IadeServisi
{
    public void Iade(IIadeEdilebilir yontem, decimal tutar) => yontem.IadeEt(tutar);
}
```

BİLGİ **.NET'in kendi içindeki bir LSP uzlaşması**

`List<T>.AsReadOnly()` metodunun döndürdüğü `ReadOnlyCollection<T>`, `ICollection<T>` arabirimini uygular; ancak `Add` çağrıldığında `NotSupportedException` fırlatır. .NET bu durumu `IsReadOnly` özelliğiyle yumuşatır. Modern kodda salt okunur koleksiyonlar için doğrudan `IReadOnlyCollection<T>` veya `IReadOnlyList<T>` kullanmak daha doğru bir sözleşmedir.

İPUCU **LSP ihlalinin kokuları**

- İstemci kodda `is`/`as` ile alt tip kontrolü yapılması.
- İçi boş bırakılmış veya `NotImplementedException` fırlatan override'lar.
- Üst sınıfın metodunu tamamen farklı bir anlamla yeniden yazan alt sınıflar.

Haftanın Özeti

- SOLID, değişimin maliyetini düşürmek için derlenmiş beş tasarım ilkesidir. Hedef düşük bağımlılık (loose coupling) ve yüksek uyumdur (high cohesion).
- **SRP:** Bir sınıfın tek bir değişme nedeni, yani tek bir aktörü olmalıdır. Hesaplama, kalıcılık, sunum ve bildirim gibi sorumluluklar ayrı sınıflara dağıtılır.
- **OCP:** Yeni davranış, mevcut kodu değiştirmek yerine yeni sınıflar ekleyerek kazandırılır. Soyutlama, polymorphism, Strategy, Decorator ve extension member'lar başlıca araçlardır.
- **LSP:** Alt tipler üst tiplerin davranış sözleşmesine uymalıdır. Ön koşullar güçlendirilmez, son koşullar zayıflatılmaz, değişmezler korunur.
- Aşağıdaki tablo, her prensip için sorulacak soruyu ve ihlalin tipik belirtisini özetler:

Prensip	Kendinize sorun	İhlal belirtisi
SRP	Bu sınıfı kaç farklı aktör, kaç farklı nedenle değiştirmek ister?	"Ve" ile anlatılan sınıflar, `Manager`/`Helper` isimleri
OCP	Yeni bir tür eklemek için mevcut kodu açmam gerekiyor mu?	Büyüyen `switch`/`if-else` zincirleri, shotgun surgery
LSP	Alt sınıfı üst sınıfın yerine koyduğumda istemci şaşırır mı?	Tip kontrolleri, `NotSupportedException` fırlatan override'lar

Anahtar Kavramlar

Terim	Türkçe karşılığı	Kısa açıklama
coupling	bağımlılık	Modüllerin birbirine ne kadar bağlı olduğu; düşük olması istenir
cohesion	uyum	Bir modülün içindeki öğelerin ne kadar ilişkili olduğu; yüksek olması istenir
rigidity	katılık	Bir değişikliğin zincirleme değişiklikler gerektirmesi
fragility	kırılganlık	Bir değişikliğin ilgisiz yerleri bozması
reason to change	değişme nedeni	SRP'de bir sınıfı değiştirmeyi gerektiren sorumluluk
actor	aktör (paydaş)	Aynı değişiklik talebini dile getiren kullanıcı veya birim grubu
abstraction	soyutlama	Ayrıntıları gizleyip yalnızca sözleşmeyi ortaya koyma
shotgun surgery	saçma dağınık değişiklik	Tek bir değişiklik için çok sayıda dosyaya dokunma zorunluluğu
extension method	genişletme metodu	Tipin koduna dokunmadan ona yeni metot ekleme
precondition / postcondition	ön koşul / son koşul	Metodun çalışmadan önce beklediği ve sonunda garanti ettiği durumlar
invariant	değişmez	Nesnenin yaşamı boyunca hep doğru kalması gereken kural
design by contract	sözleşmeye dayalı tasarım	Metotların ön/son koşul ve değişmezlerle tanımlanması
refactoring	yeniden düzenleme	Davranışı değiştirmeden kodun iç yapısını iyileştirme

Alıştırmalar

1. Kullanıcı kaydı yapan, e-posta biçimini doğrulayan, parolayı özetleyen (hash), hoş geldin e-postası gönderen ve log tutan tek bir `KullaniciYoneticisi` sınıfı yazın. Ardından sınıfı SRP'ye göre ayrıştırın ve her yeni sınıfın aktörünü belirtin.
2. Tam zamanlı, yarı zamanlı ve stajyer çalışanlar için `switch` ile maaş hesaplayan bir `MaasHesaplayici` yazın. OCP'ye uygun hâle getirin ve mevcut koda dokunmadan "sözleşmeli" çalışan tipini ekleyin.
3. `if (sekil is Daire) ... else if (sekil is Ucgen) ...` biçiminde çalışan bir `AlanHesaplayici` sınıfını polymorphism kullanarak yeniden yazın.
4. `Kus` sınıfında `Uc()` metodu varken `Penguen : Kus` tasarımının neden LSP ihlali olduğunu açıklayın ve alternatif bir hiyerarşi önerin.

5. Dikdörtgen/Kare örneğini çalıştırın, hatayı gözlemleyin; ardından değişmez `record` tabanlı çözümle yeniden yazıp aynı testin artık neden anlamsızlaştığını tartışın.
6. .NET'te `Stream.CanSeek`, `Stream.CanWrite` ve `ICollection<T>.IsReadOnly` özelliklerinin hangi LSP problemini yumuşatmak için var olduğunu açıklayın.
7. Kaynak koduna dokunmadan, C# 14 extension blokları ile `DateTime` tipine `HaftaSonuMu` adında bir extension property ekleyin.

Ek Okuma Kaynakları

- Mimari ilkeler — .NET mimari kılavuzu
- Robert C. Martin — The Single Responsibility Principle
- Robert C. Martin — The Open Closed Principle
- Extension members — C# programlama kılavuzu
- C# 14'teki yenilikler: extension members
- Makale: B. Liskov, J. Wing — *A Behavioral Notion of Subtyping*, ACM TOPLAS, 1994.
- Kitap: Robert C. Martin — *Agile Software Development: Principles, Patterns, and Practices*, Prentice Hall, 2002.